

CCF-GESP C++四级

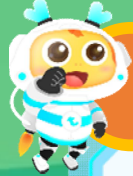
编程能力等级认证



第十四课

排序算法 2

01 插入排序

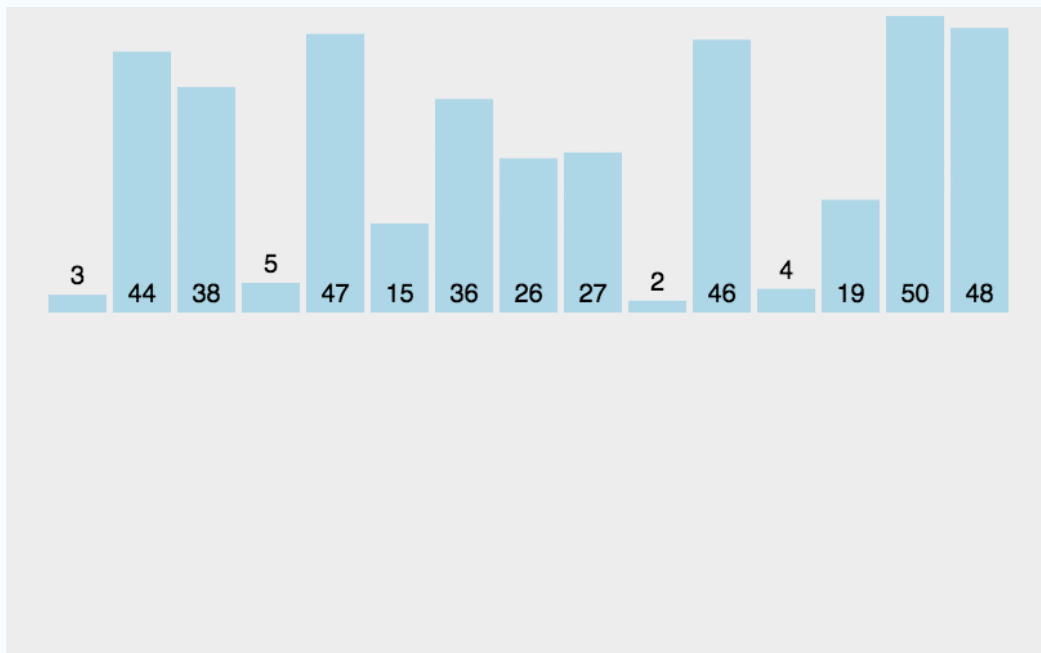


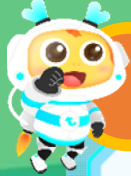
知识讲解

插入排序（升序）



将待排序的数字依次**插入**到已**有序的数列**中，并**仍然**保持有序。





知识讲解

对数组中的n个元素进行升序排序

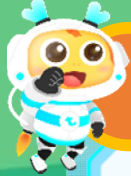
内层循环

外层循环

i	比较的数据
0	-
1	a[0]
2	a[1] a[0]
3	a[2] a[1] a[0]
4	a[3] a[2] a[1] a[0]
...	...
n-1	a[n-2] a[n-3] a[n-4] ... a[0]

$i-1 \sim 0$

```
for(int i=0;i<n;i++){  
    int x=a[i];  
    int j;  
    for(j=i-1;j>=0;j--){  
        升序 if(a[j]>x) a[j+1]=a[j];  
            else break;  
    }  
    a[j+1]=x;  
}
```



知识讲解

对数组中的n个元素进行降序排序

内层循环

外层循环

i	比较的数据
0	-
1	a[0]
2	a[1] a[0]
3	a[2] a[1] a[0]
4	a[3] a[2] a[1] a[0]
...	...
n-1	a[n-2] a[n-3] a[n-4] ... a[0]

$i-1 \sim 0$

```
for(int i=0;i<n;i++){  
    int x=a[i];  
    int j;  
    for(j=i-1;j>=0;j--){  
        降序 if(a[j]<x) a[j+1]=a[j];  
            else break;  
    }  
    a[j+1]=x;  
}
```



课堂练习

判断题

插入排序，是指将数据按照一定的顺序一个一个的插入到有序的表中，最终得到的序列就是已经排序好的数据。

☒ 正确

错误 ☐



课堂练习

选择题

若对 n 个元素进行插入排序的过程中，共需进行（ **C** ）趟。

- A. n
- B. $n+1$
- C. $n-1$
- D. $2n$



课堂练习

判断题

使用插入排序算法对序列18, 23, 19, 9, 23, 15 进行升序排序, 第3趟排序后的结果为9, 18, 19, 23, 23, 15。

☒ 正确

错误 ☐

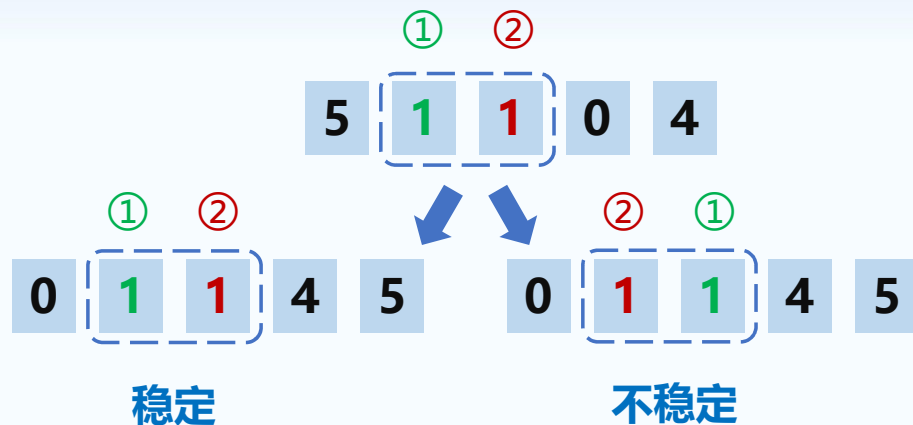
04 排序算法的稳定性



知识讲解

排序算法稳定性

在待排序数据中存在多个相同的数，**排序后**这些相同的数**先后顺序**能够保持不变，则称排序算法是**稳定的**；否则称为不稳定的。





知识讲解

排序算法对比

排序算法	稳定性
选择排序	不稳定
冒泡排序	稳定
插入排序	稳定
快速排序	不稳定
归并排序	稳定



真题解析

判断题

(样题) 冒泡排序是一种稳定的排序算法。

☒ 正确

错误 ☐



真题解析

选择题

(样题) 排序算法的稳定性是指 (**A**)

- A. 相同元素在排序后的相对顺序保持不变
- B. 排序算法的性能稳定
- C. 排序算法对任意输入都有较好的效果
- D. 排序算法容易实现



真题解析

选择题

(2023年6月) 排序算法是稳定的 (Stable Sorting) , 就是指排序算法可以保证, 在待排序数据中有两个相等记录的关键字 R 和 S (R 出现在 S 之前) , 在排序后的列表中 R 也一定在 S 前。下面关于排序稳定性的描述, 正确的是 (C)

- A. 冒泡排序是不稳定的。
- B. 插入排序是不稳定的。
- C. 选择排序是不稳定的。
- D. 以上都不正确。



课堂练习

插入排序

【问题描述】

插入排序是一种非常常见且简单的排序算法。小 Z 是一名大一的新生，今天H老师刚刚在上课的时候讲了**插入排序**算法。

假设比较两个元素的时间为 $O(1)$ ，则插入排序可以 $O(n^2)$ 的时间复杂度完成长度为 n 的数组的排序。不妨假设这 n 个数字分别存储在 $a_1, a_2, \dots, a_n$ 之中，则如下伪代码给出了插入排序算法的一种最简单的实现方式。

这下面是 C/C++ 的示范代码：

```
for (int i = 1; i <= n; i++)  
    for (int j = i; j >= 2; j--)  
        if (a[j] < a[j-1]){  
            int t = a[j-1];  
            a[j-1] = a[j];  
            a[j] = t;  
        }
```




课堂练习

插入排序

【问题描述】

为了帮助小 Z 更好的理解插入排序，H老师留下了这么一道家庭作业：
H老师给了一个长度为 n 的数组 a ，数组下标从1开始，并且数组中的所有元素均为非负整数。
小 Z 需要支持在数组 a 上的 Q 次操作，操作共两种，参数分别如下：

- ① $x\ v$ ：这是第一种操作，会将 a 的第 x 个元素，也就是 a_x 的值，修改为 v 。保证 $1 \leq x \leq n$, $1 \leq v \leq 10^9$ 。注意这种操作会改变数组的元素，修改得到的数组会被保留，也会影响后续的操作。
- ② x ：这是第二种操作，假设H老师按照上面的伪代码对 a 数组进行排序，你需要告诉H老师原来 a 的第 x 个元素，也就是 a_x ，在排序后的新数组所处的位置。保证 $1 \leq x \leq n$ 。注意这种操作不会改变数组的元素，排序后的数组不会被保留，也不会影响后续的操作。

H 老师不喜欢过多的修改，所以他保证类型 1 的操作次数不超过 5000。
小 Z 没有学过计算机竞赛，因此小 Z 并不会做这道题。他找到了你来帮助他解决这个问题。



课堂练习

插入排序

【输入格式】

第一行，包含两个正整数 n, Q ，表示数组长度和操作次数。

第二行，包含 n 个空格分隔的非负整数，其中第 i 个非负整数表示 a_i 。

接下来 Q 行，每行 2~ 3 个正整数，表示一次操作，操作格式见【题目描述】。

【输出格式】

对于每一次类型为 2 的询问，输出一行一个正整数表示答案。

【输入样例】

```
3 4
3 2 1
2 3
1 3 2
2 2
2 3
```

对于所有测试数据，满足 $1 \leq n \leq 8000$ ，
 $1 \leq Q \leq 2 \times 10^5$ ， $1 \leq x \leq n$ ， $1 \leq v, a_i \leq 10^9$ 。

【输出样例】

```
1
1
2
```



课堂练习

思路分析

【输入样例】

3个数

3 4

3次操作

3 2 1

输入的3个数

2 3

表示原序列第三个位置

1 3 2

原序列第三个位置的值修改为2

2 2

2 3

2: 表示**查询**
查询前要先**稳定**排序(升序)

1: 表示**修改**

3 2 1

下标: 1 2 3

第一次操作: 1. **排序** 1 2 3 2. 查询 1

下标: 1 2 3

第二次操作: **修改** 3 2 2

下标: 1 2 3

第三次操作: 1. **排序** 2 2 3 2. 查询 1

下标: 1 2 3

第四次操作: 1. **排序** 2 2 3 2. 查询 2

下标: 1 2 3



课堂练习

初始化相关操作

- 定义**结构体数组****a**
- 定义**整型变量****n**，表示**n**个数字；定义**整型变量****q**，保存操作次数

3 2 1

下标: 1 2 3

```
const int MAXN=8005;
```

题目中n值最大为8000

```
struct node{  
    int val,id;  
} a[MAXN];
```

val为数据，id为下标

```
int main(){  
    int n,q;  
    scanf("%d%d",&n,&q);  
    for(int i=1;i<=n;i++){  
        scanf("%d",&a[i].val);  
        a[i].id=i;  
    }  
}
```




课堂练习

对初始数据进行稳定排序

定义**cmp**排序函数，对node结构体进行**稳定**排序
对数组a进行**稳定**排序（升序）

排序后 1 2 3

下标: 1 2 3

```
bool cmp(node x, node y) {  
    if (x.val != y.val)      val升序  
        return x.val < y.val;  
    return x.id < y.id;  
}  
int t[MAXN];  
int main() {  
    ...  
    sort(a+1, a+n+1, cmp);  
    for (int i=1; i<=n; i++)  
        t[a[i].id] = i;  
}
```



课堂练习

对初始数据进行稳定排序

把排序后的元素位置，记录在数组t中

原数组a: 3 2 1 排序后 1 2 3
: 下标: 1 2 3

数组t: 1 2 3 元素新位置
下标: 3 2 1 元素原位置

```
bool cmp(node x, node y) {  
    if (x.val != y.val)  
        return x.val < y.val;  
    return x.id < y.id;  
}  
int t[MAXN];  
int main() {
```

```
    ...  
    ➔ stable_sort(a+1, a+n+1, cmp);  
    for (int i=1; i<=n; i++)  
        t[a[i].id] = i;  
        元素原位置      元素新位置
```



课堂练习

执行Q次操作

循环 q 次，输入操作选项，根据 opt 值分支判断

```
for(int i=1;i<=q;i++){  
    int opt;  
    scanf("%d",&opt);  
    if(opt==2){  
        //查询操作  
    }else if(opt==1){  
        //修改操作  
    }  
}  
return 0;  
}
```



课堂练习

如果操作选项为2，则进行查询操作

输入查询位置，输出该位置元素对应的新位置

原数组a: 3 2 1  排序后 1 2 3

下标: 1 2 3

数组t: 1 2 3  元素新位置

下标: 3 2 1  元素原位置

第一次操作:

2 3



```
if(opt==2){  
    scanf("%d",&x);  
    printf("%d\n",t[x]);  
}  
else if(opt==1){  
    //修改操作  
}  
return 0;  
}
```

元素原位置

访问数组t



课堂练习

如果操作选项为1，则进行修改操作

① 定义整型变量 x 和 v ，分别保存位置和修改值

→

```
if(opt==1){  
    int x, v;  
    scanf("%d%d", &x, &v);  
    a[t[x]].val = v;  
    for(int j=t[x];j>=2;j--){  
        if(cmp(a[j],a[j-1])){  
            node k=a[j];  
            a[j]=a[j-1];  
            a[j-1]=k;  
        }  
    }  
}
```

元素原位置



课堂练习

如果操作选项为1，则进行修改操作

② 修改原位置上的值

原数组a: 3 2 1
下标: 1 2 3

排序后: 2 2 3

数组t: 1 2 3
下标: 3 2 1

元素新位置
元素原位置

第二次操作:

1 3 2
元素原位置

```
if(opt==1){  
    int x, v;  
    scanf("%d%d", &x, &v);  
    a[t[x]].val = v;  
    for(int j=t[x];j>=2;j--)  
        if(cmp(a[j],a[j-1])){  
            node k=a[j];  
            a[j]=a[j-1];  
            a[j-1]=k;  
        }  
}
```



课堂练习

如果操作选项为1，则进行修改操作

③ 对数组a，从当前位置向前进行插入排序

原数组a: 3 2 1
下标: 1 2 3

排序后: 2 2 3

待排序元素

```
if(opt==1){  
    int x, v;  
    scanf("%d%d", &x, &v);  
    a[t[x]].val = v;  
    for(int j=t[x];j>=2;j--){  
        if(cmp(a[j],a[j-1])){  
            node k=a[j];  
            a[j]=a[j-1];  
            a[j-1]=k;  
        }  
    }  
}
```

循环不执行



课堂练习

如果操作选项为1，则进行修改操作

④ 对数组a，从当前位置向后进行冒泡排序

原数组a: 3 2 1
下标: 1 2 3

排序后: 2 2 3
id: 3 2 1

待排序元素

值相同时，按
id升序排序

排序后: 2 2 3
id: 2 3 1

```
if(opt==1){
    int x, v;
    scanf("%d%d", &x, &v);
    a[t[x]].val = v;
    ...
    for(int j=t[x];j<n;j++){
        if(cmp(a[j+1],a[j])){
            node k=a[j];
            a[j]=a[j+1];
            a[j+1]=k;
        }
    }
    for(int i=1;i<=n;i++){
        t[a[i].id]=i;
    }
}
```



课堂练习

如果操作选项为1，则进行修改操作

④ 对数组a，从当前位置向后进行冒泡排序

排序后: 2 2 3
id: 2 3 1

数组t 1 2 3 元素新位置
下标: 2 3 1 元素原位置

```
if(opt==1){  
    int x, v;  
    scanf("%d%d", &x, &v);  
    a[t[x]].val = v;  
    ...  
    for(int j=t[x];j<n;j++){  
        if(cmp(a[j+1],a[j])){  
            node k=a[j];  
            a[j]=a[j+1];  
            a[j+1]=k;  
        }  
    }  
    for(int i=1;i<=n;i++){  
        t[a[i].id]=i;  
    }  
}
```




课堂练习

如果操作选项为1，则进行修改操作

⑤ 再次进行查询操作

排序后: 2 2 3
id: 2 3 1

第三次操作:
2 2

数组t: 1 2 3
下标: 2 3 1

元素新位置
元素原位置

```
if(opt==1){  
    int x, v;  
    scanf("%d%d", &x, &v);  
    a[t[x]].val = v;  
    ...  
    for(int j=t[x];j<n;j++){  
        if(cmp(a[j+1],a[j])){  
            node k=a[j];  
            a[j]=a[j+1];  
            a[j+1]=k;  
        }  
    }  
    for(int i=1;i<=n;i++){  
        t[a[i].id]=i;  
    }  
}
```




课堂练习

【完整代码】

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN=8005;
int n,q;
int t[MAXN];
struct node
{
    int val,id;
} a[MAXN];
bool cmp(node x,node y)
{
    if(x.val!=y.val) return x.val<y.val;
    return x.id<y.id;
}
```

```
int main()
{
    scanf("%d%d",&n,&q);
    for(int i=1; i<=n; i++)
    {
        scanf("%d",&a[i].val);
        a[i].id=i;
    }
    sort(a+1,a+n+1,cmp);
    for(int i=1; i<=n; i++)
        t[a[i].id]=i;
}
```



课堂练习

【完整代码】

```
for (int i = 1; i <= q; i++) {  
    int opt, x, v;  
    scanf("%d", &opt);  
    if (opt == 1) {  
        scanf("%d%d", &x, &v);  
        a[t[x]].val = v;  
        for (int j = t[x]; j >= 2; j--)  
            if (cmp(a[j], a[j - 1])) {  
                node k = a[j];  
                a[j] = a[j - 1];  
                a[j - 1] = k;  
            }  
    }  
}
```

```
for (int j = t[x]; j < n; j++)  
    if (cmp(a[j + 1], a[j])) {  
        node k = a[j];  
        a[j] = a[j + 1];  
        a[j + 1] = k;  
    }  
for (int i = 1; i <= n; i++)  
    t[a[i].id] = i;  
} else {  
    scanf("%d", &x);  
    printf("%d\n", t[x]);  
}  
}  
return 0;  
}
```